

# Pixel Battle — Timeline Script (Sub-project D) — Design Spec

## Pixel Battle — Timeline Script (Sub-project D) — Design Spec

For agentic workers: This is a *design spec*. The step-by-step implementation plan is generated separately via `superpowers:writing-plans`.

- **Date:** 2026-05-24
- **Status:** Design approved by user — full autonomous execution granted (no approval gates), runs under `/loop`.
- **Relationship:** Sub-project D of the pixel\_battle combat work, following A (combat-feel polish), B (scripted combat), and C (ranged combat & mobility).

### 1. Motivation

After Sub-project C, all 5 fight scripts render to decisive KO, but the user observed residual choppiness on screen. Diagnosis: the current condition-driven `ScriptDriver` uses `{do, until}` intents, and when a condition cannot be satisfied (e.g. `until: skill_done` for an attack that mis-positions and no-ops, or `until: dist<=110` when the target Flash-escapes), the intent stalls until the implicit `INTENT_MAX_MS = 4000ms` timeout fires. A 4-second “wait for nothing” mid-fight reads as the character standing still — the 卡卡 the user sees.

The user proposed a different authoring model: write the fight as a “high-energy combat novel” — prose narrating who does what at minute X second Y — then play it on top of the existing physics engine. Absolute timestamps, no conditional waits. Engine handles consequences (physics, hits, status effects); the script dictates intent.

## 2. Goal

Replace condition-driven intents with an **absolute-timeline** action source. Each fight becomes a YAML file with two parallel timelines (one per character), each timeline a sequence of {t, do} events fired by clock. Prose narration lives as YAML comments — it is the human-readable / LLM-friendly half, and the YAML body is the engine-readable half. Convert the 5 existing condition scripts to this new format using a trace-dumping tool that records the actions emitted by the current driver, then hand- or LLM-polish the trace into a final timeline with novel-style prose.

## 3. Scope & Constraints

**In scope:** 1. **TimelineDriver** — a new action source that fires per-character events on absolute time. 2. **Timeline YAML format** — {t, do} events with prose annotations as comments; loader detects and dispatches to the new driver. 3. **Trace tool** (scripts/dump\_timeline.py) — runs an existing condition script through the engine and outputs the per-action timestamp trace. 4. **Convert all 5 existing scripts** to the new format; old condition YAMLs archived to pixel\_battle/data/scripts/legacy/.

**Hard constraint: No engine changes.** Physics, skills, hit resolution, hitstop, status effects, renderer, camera — all unchanged. Only the action source changes.

**Hard constraint: No RL retraining.** Same as Sub-projects B/C — RL is sidelined for scripted videos; this sub-project does not touch the RL action space or the trained checkpoint.

**Out of scope (deferred):** – Camera control via timeline (climax zoom, KO slow-mo, screen shake cues). – Conditional branches in the timeline (no if target\_hp<50 then jump\_to chapter:finale). – LLM auto-generation of complete prose+timeline from a high-level pitch. – Relative timestamps (+0.5s after previous event); all timestamps are absolute. – Character voice / dialogue bubbles / on-screen text.

## 4. Current state (verified)

- pixel\_battle/script/driver.py::ScriptDriver reads condition-based YAML and emits per-tick actions via next\_actions(battle, elapsed\_ms) -> (left\_action, right\_action).
- pixel\_battle/rl/play.py::\_render\_fight is action-source-agnostic — it calls action\_source(env) once per tick. Both the RL path (PPO wrapper)

and the scripted path (`ScriptDriver`) flow through here.

- Engine `Battle.tick_ms(dt_ms)` advances one tick; `elapsed_ms` tracks simulation time and is **paused during hitstop** (Sub-project A).
- The 11 do-verbs in `pixel_battle/script/loader.py::D0_VERBS` map to engine action ints 0–10 (incl. `flash:in/flash:back` from Sub-project C).
- Match timeout: `MATCH_TIMEOUT_MS = 60000`. Intent timeout: `INTENT_MAX_MS = 4000`. The latter is the root cause of the 4-second stutter; the new driver simply does not use it.

## 5. Architecture — TimelineDriver as the action source

The engine and `_render_fight` are unchanged. A new action source plugs into the existing interface:

- **TimelineDriver** (`pixel_battle/script/timeline_driver.py`): holds two parallel event lists (one per character), per-character cursors, per-character accumulated delay offsets. Each tick:
  - For each character, peek the next event in that character's list.
  - If `elapsed_ms >= event.t + char_delay_offset` AND the character can act, emit `event.action_int` and advance the cursor.
  - If the scheduled time has come but the character cannot act, increment `char_delay_offset` by one engine tick (`ENGINE_TICK_MS`) and emit `idle (0)`. The event will retry next tick.
  - If the scheduled time has not yet come, emit `idle`.
  - If the cursor is past the end of the list, emit `idle (script exhausted)`.
- **TimelineLoader** (`pixel_battle/script/timeline_loader.py`): parses the new YAML format, ignores prose-only comments, validates verbs and skill IDs.
- **Loader dispatch**: `pixel_battle/script/loader.py` (or a new top-level entry point) auto-detects format — presence of `left_timeline/right_timeline` keys → `TimelineDriver`; presence of `left_script/right_script` keys → legacy `ScriptDriver` (unchanged, still works for the archived files).
- **play\_scripted.py**: unchanged — it calls the loader, which returns the right driver type.
- The legacy `ScriptDriver` and the 5 old condition YAMLs are preserved (moved to `pixel_battle/data/scripts/legacy/`). Future cleanup may delete them; this sub-project does not.

## 6. Timeline YAML format

```
name: "拉克絲 vs 蓋倫 — 風箏與追擊"
left: garen
right: lux
duration_ms: 18000    # author-declared expected fight length
```

```
# —— 散文劇本(註解、人讀 / LLM 讀) ——
# 【第一幕 · 開場 0.0–3.0s】
# 蓋倫低吼一聲, 大劍出鞘, 身形如鐵塊般壓進。
# 拉克絲足尖一點向後輕掠, 光之法杖泛起淡金 — 她需要距離。
# 【第二幕 · 蓄勢 3.0–7.5s】
# 首發《光之束縛》如鎖鏈般射出 ...
```

```
left_timeline:
  - {t: 0,      do: "idle"}
  - {t: 500,    do: "advance"}
  - {t: 3200,   do: "attack:basic"}
  - {t: 4500,   do: "flash:in"}
  - {t: 5000,   do: "cast:decisive_strike"}
  - {t: 8500,   do: "cast:judgment"}
  - {t: 13000,  do: "cast:demacian_justice"}

right_timeline:
  - {t: 0,      do: "idle"}
  - {t: 800,    do: "retreat"}
  - {t: 3000,   do: "cast:light_binding"}
  - {t: 5500,   do: "flash:back"}
  - {t: 6000,   do: "retreat"}
  - {t: 8000,   do: "cast:lucent_singularity"}
```

**Field rules:** – t — non-negative integer, milliseconds since match start (elapsed\_ms = 0). – do — one of the 11 existing do-verbs (idle, retreat, advance, jump, attack:basic, attack:cd, attack:special, attack:ultimate, attack:kick, flash:in, flash:back), OR cast:<skill\_id> to name a specific skill on the character. – cast:<skill\_id> — the loader looks up the named skill in characters.json for the character whose timeline this is, determines its SkillType (basic/cd/ultimate/special/kick), and routes to the matching engine action. The driver also stashes the skill id so the engine's \_start\_attack\_with\_kind can select it directly (bypassing affordable[0]). If the named skill is not on the character, the loader rejects the file. – Each event in left\_timeline / right\_timeline is required to be in strictly increasing t order — the loader rejects out-of-order events. – duration\_ms is the author's expected total length; the driver does not enforce it (the engine's MATCH\_TIMEOUT\_MS is the hard cap), but it is used by the integration test for a “render duration within ±20 % of authored” sanity check. – All prose lives in # comments. YAML parsers strip comments; the driver sees only the structured fields.

## 7. Conflict policy — events vs. engine reality

When an event's scheduled time arrives but the character cannot act, the event waits. The character's accumulated delay grows, and **all subsequent events on that character's timeline shift forward by the same delay** (since they are scheduled relative to `event.t + char_delay_offset`). The two timelines are independent — a delay on LEFT does not slide RIGHT's timeline.

**“Cannot act” predicate `_can_act(char, verb)`** — kept deliberately small; the engine's `_apply_action` remains the authority on cooldown / mp / range / facing, so the driver only needs to detect the two cases where the engine would obviously waste the event:

- `char.action_state` in `{ATTACK_WINDUP, ATTACK_STRIKE, ATTACK_RECOVER}` and `verb != "idle"` → cannot act (a new attack/movement is silently dropped mid-animation; the script's intent is to wait until the previous attack finishes, not to no-op forever).
- `char.has_effect("root")` and `verb` in `{"advance", "retreat", "jump"}` → cannot act (movement under root is engine-no-op; the script's intent is to wait for root to expire). Casting and Flash are NOT blocked by root — Flash is a teleport that bypasses root, and cast events should fire on time so the character starts windup the moment root drops.

Everything else (cooldown not ready, mp insufficient, out of range, mid-jump for ground actions) the engine handles silently via `_apply_action` — the cursor still advances, and the script-authoring flow tolerates that (the trace tool in §8 surfaces misses for the author to retime). Hitstop is engine-side — `tick_ms` no-ops during hitstop, `elapsed_ms` does not advance, so the driver naturally pauses with it. No special handling.

**KO mid-script:** The engine ends the match on  $HP \leq 0$  (`battle.terminated = True`). The `_render_fight` loop breaks out; the driver is not called further. Events past the KO point are simply unfired — fine.

**Script exhausted before KO:** Both characters' cursors past the end → both idle → engine `MATCH_TIMEOUT_MS` (60 s) eventually ends the match. If `duration_ms` was authored close to the actual KO time, this rarely fires.

## 8. Conversion strategy — `dump_timeline.py`

A small one-off tool: `pixel_battle/scripts/dump_timeline.py`. Given a legacy condition YAML, it:

1. Loads the file with the legacy ScriptDriver.
2. Builds the same env / Battle that play\_scripted.py would build.
3. Runs the simulation tick-by-tick, calling driver.next\_actions(...) each tick.
4. Each tick where either character emits a non-idle action, append {t: elapsed\_ms, char: left|right, do: <verb-or-cast>} to a trace.
5. Output the trace as a candidate timeline YAML, with the legacy file's name, left, right carried over, and a duration\_ms set to the actual KO time (or MATCH\_TIMEOUT\_MS if the legacy script timed out).

The output is a **starting point**, not the final artifact: – Timestamps land on raw tick boundaries — the author rounds them to the nearest 100 ms for readability. – Reactive scripts that produced misses (e.g. C-G3's Glass Slab attack-at-230-px miss) will show those misses in the trace. The author trims or re-times them. – The author adds the prose chapter headers (the novel-style narration in # comments) on top.

All 5 legacy scripts get this treatment. The trace tool is one-off — once the 5 timelines are authored, it's not needed for routine work (but stays in the repo for future conversions).

## 9. Error handling

- **Loader rejects** at load time: unknown do verb, unknown skill id in cast: <id>, missing left/right/left\_timeline/right\_timeline, events not in strictly increasing t order, negative t, non-integer t, unknown character id.
- **cast:<id> for a skill not on the character** → load-time error with the message naming the skill and the character.
- An event firing for a character mid-action becomes a delayed event (see §7); it never crashes.
- A duration\_ms mismatch with the actual render duration is a warning in the integration test, not an error — the author tunes the field if needed.

## 10. Testing

- **TimelineDriver:** event fires when elapsed\_ms reaches event.t (no earlier); event delays when can\_act is false and emits idle; subsequent events on the same character shift by the accumulated delay; exhausted cursor emits idle; the two timelines are independent (LEFT delay does not shift RIGHT events).
- **cast:<skill\_id>:** correctly resolves to the right action int by the skill's SkillType; engine's \_start\_attack\_with\_kind (or equivalent) actually

selects the named skill rather than `affordable[0]`.

- **TimelineLoader:** parses a valid YAML; ignores prose comments; rejects each malformed case with a clear error message.
- **dump\_timeline.py:** runs against each of the 5 legacy scripts and produces a parseable timeline trace; the trace has one event per non-idle action emitted by the legacy run; timestamps are monotonically non-decreasing.
- **Integration:** each of the 5 converted timeline scripts loads, renders, reaches a decisive KO (engine terminated=True, loser HP=0), and the rendered duration is within  $\pm 20\%$  of the file's authored `duration_ms`.
- **Smoothness regression:** for each converted script, the longest run of consecutive ticks on either character's timeline where the driver emits idle (action 0) is **< 1000 ms**. This is the direct test of the user's “卡卡” complaint — the legacy `INTENT_MAX_MS = 4000ms` stalls would fail it by 4x.
- All pre-existing tests (~381) stay green.

## 11. Validation

Re-render all 5 converted timeline scripts via `play_scripted.py`. Inspect each mp4: – Visually compare smoothness against the Sub-project C renders — no more 4-second “wait for nothing” stalls. – All 5 still reach decisive KO. – Total fight length matches the file's `duration_ms` (with the natural variance from engine determinism + hitstop). – The prose chapter notes in the YAML read as a coherent fight narrative.

## 12. Future work

- **Camera timeline** — `camera_timeline: [{t: 13000, do: zoom_in, target: lux, scale: 1.4}, {t: 14500, do: slow_mo, factor: 0.5}]` for climax framing and KO slow-motion. Probably the next sub-project after D ships.
- **Conditional branches** — if `target_hp < 50` then `jump_to chapter:finale` for adaptive pacing without losing absolute-time predictability.
- **LLM auto-generation** — given a pitch (“拉克絲打蓋倫, 30 秒, 法師勝”), an LLM writes the full prose+YAML in one shot; the trace tool becomes one possible bootstrap, not the only one.
- **Relative timestamps** — `{t: "+500", ...}` syntactic sugar for “0.5 s after the previous event on this timeline.”

- **Voice / dialogue** — `dialogue_timeline`: [{t: 3000, char: garen, line: "為了狄馬西亞!"}] paired with ElevenLabs or similar.